



PROJECT 4 SPECIFICATION DOCUMENT

PROJECT 4 Recommendation and Streaming
Platform Service

JiaJiao Xu, Jordi Vidal
GROUP 6

Contents

Introduction..... 2

Project Objectives and Requirements 4

 Objectives..... 4

 Functional Requirements 5

 Non-Functional Requirements..... 6

Data Collection 7

 Movie Dataset 7

 Data cleaning movie dataset 10

 User Dataset 13

 User-Movie Interactions Dataset..... 14

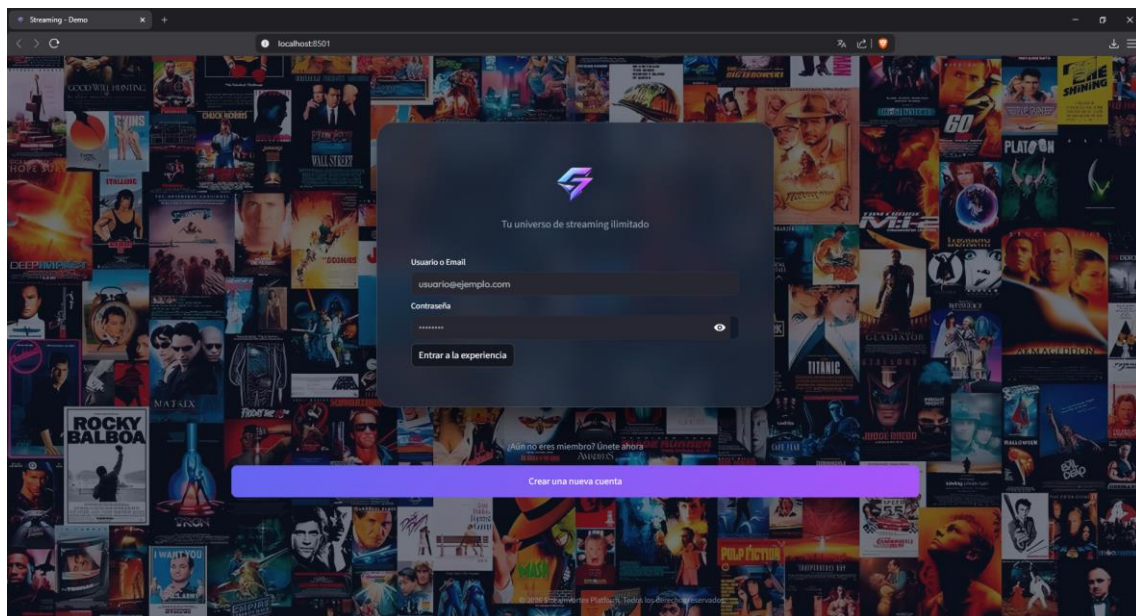
Introduction

The objective of this project was to design and develop a software system focused on movie recommendation and rating management for users seeking personalized film suggestions based on their preferences and interactions.

The main problem addressed is the difficulty users face when trying to discover movies that match their personal tastes among a large and constantly growing catalog of available content. Traditional recommendation methods are often too generic and do not adapt dynamically to user feedback and changing preferences.

The developed solution allows the system to automatically analyze user interactions, ratings, likes, and dislikes in order to build a personalized preference profile. Based on this profile, the application generates tailored movie recommendations using different recommendation models, including content-based filtering, similarity analysis, and embedding-based semantic recommendations.

Additionally, the platform provides an interactive interface where users can explore movies, view detailed information, rate films, and immediately influence future recommendations. This creates a dynamic recommendation process in which the system continuously updates the user's preferences and improves the quality and relevance of the suggested content.



Streaming - Demo

localhost:8501/?page=register



Crea tu cuenta y empieza a explorar

DATOS DE CUENTA

Nombre de usuario:

Correo electrónico:

Contraseña:

Confirmar contraseña:

PERFIL PERSONAL

Género: ☒ Hombre ☐ Mujer ☐ Otro

Fecha de nacimiento:

GÉNEROS FAVORITOS (Máx. 4)

Elige al menos un género...

Crear mi cuenta +

Volver a inicio de sesión

Streaming - Demo

localhost:8501/?page=dashboard



test07

Películas

Audit

Salir

Plataforma de recomendación de películas

Buscar película por título exacto y pulsa Enter...

Recomendaciones + Películas








Streaming - Demo

localhost:8501/?page=moviecard&tmdb_id=106646



test07

Películas

Audit

Ver photos

The Wolf of Wall Street

Tráiler **FIND** **IMDb** **8.0**

A New York stockbroker refuses to cooperate in a large securities fraud case involving corruption on Wall Street, corporate banking world and mob infiltration. Based on Jordan Belfort's autobiography.

Estreno: 2013-12-25

Género: Crime, Drama, Comedy

Popularidad: 17

Nota media: 8.0 / 10

Duración: 180 min

Votos: 25,544

TMDB ID: 106646

Año: 2013

TV VALORACIÓN

☆☆☆☆

Aún no has valorado esta película

Project Objectives and Requirements

Objectives

The main objective of this project is to develop a movie recommendation platform that provides users with a personalized and efficient experience by using data analysis and Big Data techniques to manage large volumes of information and generate recommendations adapted to their preferences and ratings.

More specifically, the project pursues the following objectives:

- Design a system capable of storing, managing, and analyzing information related to movies, users, and interactions.
- Allow users to browse the available movie catalog and access detailed information about each title.
- Incorporate a rating system so users can evaluate the movies they have watched.
- Use user ratings and interactions to build and continuously update their preference profile.
- Generate personalized movie recommendations based on user interaction history and similarity between movies.
- Apply recommendation techniques based on content filtering, semantic similarity, and embeddings to improve the accuracy of suggestions.
- Design a scalable solution capable of adapting to catalog growth and increasing numbers of users and interactions.
- Integrate monitoring and performance analysis mechanisms to supervise system behavior and detect possible issues.
- Provide an intuitive, dynamic, and interactive user experience that allows simple and efficient navigation through the application.

Functional Requirements

The system must meet the following functional requirements:

- Allow access to the application and navigation through the movie catalog.
- Display a list of movies with basic information such as title, genre, year, rating, and description.
- Allow users to view complete detailed information for each movie.
- Allow users to rate movies through a scoring system.
- Record user interactions, including positive and negative ratings.
- Update the user preference profile based on submitted ratings.
- Generate personalized recommendations according to the user's preference history.
- Display similar movies related to a specific movie selected by the user.
- Recalculate recommendations whenever the user modifies their ratings.
- Prioritize recommendations that better match the user's current profile.
- Allow recommendation results to be consulted both from the main dashboard and from individual movie detail pages.
- Correctly manage data loaded from the database or input files.
- Display warning or error messages when issues occur during data loading or processing.

Non-Functional Requirements

The system must satisfy the following non-functional requirements:

- **Usability:** the application must be intuitive, clear, and easy to use for users without technical knowledge.
- **Performance:** the system must provide adequate response times even when processing large numbers of movies or user interactions.
- **Scalability:** the solution must support growth in users, movies, and stored records without losing functionality.
- **Maintainability:** the architecture and code must facilitate future review, modification, and expansion.
- **Portability:** the application must be executable in different environments with reasonable configuration effort.
- **Robustness:** the system must correctly handle data errors, loading failures, and invalid input situations.
- **Reproducibility:** given the same input data, the system must generate consistent and comparable results.
- **Efficiency:** recommendation processing must optimize resource usage to avoid unnecessary delays.
- **Traceability:** user interactions must be properly recorded to allow behavior analysis and continuous system improvement.

Data Collection

Description and Design of the Data Management and Storage System

The data management and storage system for this movie recommendation project is designed around a modular architecture based on three independent datasets: **users**, **user-movie interactions**, and **movies**. This separation follows a clear design principle where each dataset represents a different level of information: user identity and preferences, user behavior, and movie content metadata.

This structure improves maintainability, clarity, and scalability, as each dataset can evolve independently without affecting the others. It also allows the recommendation system to combine different types of information efficiently when building user profiles and generating recommendations.

The **user dataset** is generated using a custom script called **userdatagenerator.py** and defines the static profile of each user. The **user-movie interactions dataset** is generated using **userinteraction_generator.py** and stores the behavioral data required to infer preferences. The **movie dataset** is built using a dedicated extraction script that connects to the **TMDb API**, followed by a data cleaning process to ensure quality and consistency.

The final system integrates these three datasets to support a hybrid recommendation approach that combines user preferences, behavioral patterns, and rich movie metadata.

Movie Dataset

For a movie recommendation and rating platform, it is important to use a dataset that includes detailed information about movies. The system not only needs user ratings but also rich movie metadata to generate accurate and personalized recommendations.

This data should include elements such as titles, genres, release dates, descriptions, keywords, ratings, and other relevant attributes that help improve recommendation quality.

Initially, the project used a dataset downloaded from an online source because it was easy to access and contained a large number of movie records. However, after analyzing it, it was found that it focused mainly on user ratings and lacked enough detailed information about the movies themselves.

Most of the available data was related to scores and user interactions, but it did not include enough descriptive content like plot details, cast, or keywords, which are important for content-based and hybrid recommendation methods.

For this reason, the dataset was analyzed using R to evaluate its structure and usefulness. The results showed that although it worked well for rating-based recommendations, it was not suitable for a more complete recommendation system.

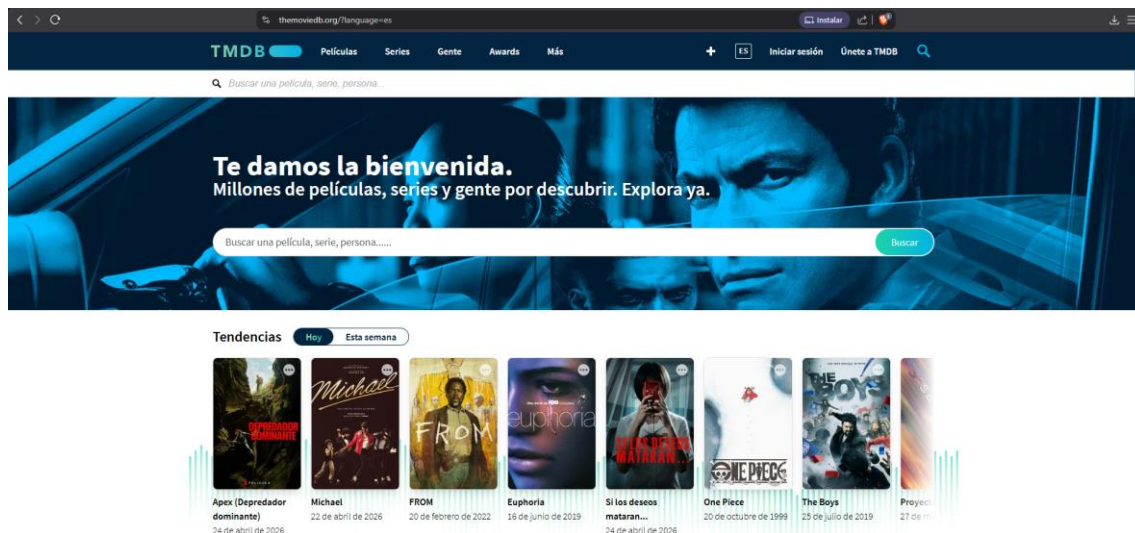
Because of this, the initial dataset was discarded and other data sources with richer movie information were explored for the final version of the project.

```
> # Cargar CSV
> datos <- read.csv("C:\\Users\\Jordi\\Desktop\\Master IABigData\\Clases en R\\datos\\datos.csv",
stringsAsFactors = FALSE)
> datos[1, ]
  movieId      title      genres
1      1 Toy Story (1995) Adventure|Animation|Children|Comedy|Fantasy
> # Cargar CSV
> datos <- read.csv("C:\\Users\\Jordi\\Desktop\\Master IABigData\\Clases en R\\datos\\datos.csv",
stringsAsFactors = FALSE)
> datos[1, ]
  movieId      title      genres
1      1 Toy Story (1995) Adventure|Animation|Children|Comedy|Fantasy
> datos <- read.csv("C:\\Users\\Jordi\\Desktop\\Master IABigData\\Clases en R\\datos\\datos.csv",
stringsAsFactors = FALSE)
> datos[1, ]
  movieId imdbId tmdbId
1      1 114709    862
> datos <- read.csv("C:\\Users\\Jordi\\Desktop\\Master IABigData\\Clases en R\\datos\\datos.csv",
stringsAsFactors = FALSE)
> datos[1, ]
  userId movieId rating timestamp
1      1      17      4  944249077
> datos <- read.csv("C:\\Users\\Jordi\\Desktop\\Master IABigData\\Clases en R\\datos\\datos.csv",
stringsAsFactors = FALSE)
> datos[1, ]
  userId movieId      tag timestamp
1     22  26479 Kevin Kline 1583038886
> |
```

Another option that was considered, and the one that was finally chosen for the project, was the use of the API provided by the TMDb (The Movie Database) website.

This API offered a much more complete source of movie information, including titles, genres, descriptions, release dates, posters, ratings, popularity data, and other important metadata needed for the recommendation system.

Unlike the initial dataset, the TMDb API allowed access to updated and richer information, making it a better choice for building a personalized movie recommendation and rating platform.



To use the TMDb API, it is necessary to create an account on the official TMDb website and generate a personal API key. This key is required to authenticate all requests made to the API and access the movie database.

Once the API key was obtained, a data extraction script was developed to build the final dataset. The script works by iterating through movie IDs, making requests to the TMDb API, and storing the returned information. It also includes basic error handling, retry logic, and periodic saving to avoid data loss during execution.

The selected fields for the dataset were chosen based on their relevance to the recommendation system and the type of analysis required. The final dataset includes:

- **tmbld:** Unique identifier for each movie, used to avoid duplicates and ensure data consistency.
- **title:** Movie title, used for display and identification in the application.
- **genres:** Genre classification, essential for content-based filtering and similarity comparisons.
- **overview:** Text description of the movie, used for semantic analysis and embedding-based recommendations.
- **poster_url:** Image URL for the movie poster, used to improve the user interface experience.
- **vote_average** and **vote_count:** Rating information used to understand general audience perception.
- **popularity:** Metric that helps prioritize trending or frequently viewed movies.
- **runtime:** Movie duration, which can be useful for filtering or user preferences.

- `release_date`: Release information used for filtering and contextual recommendations.

These columns were selected because they provide a good balance between descriptive content (for content-based and semantic recommendations) and numerical features (for ranking and filtering). This combination allows the system to generate more accurate and personalized recommendations while also supporting a richer user experience in the interface.

After running the script for a period of time, we obtained a dataset with the following structure:

```
===== COLUMNAS DISPONIBLES =====
> print(colnames(df))
[1] "tmdbId"      "title"      "genres"     "overview"   "poster_url" "vote_average" "vote_count" "popularity" "runtime"
[10] "release_date"
>
> cat("\n\n===== EJEMPLO DE UN REGISTRO =====\n")

===== EJEMPLO DE UN REGISTRO =====
>
> # Mostrar 1 fila de ejemplo (la primera)
> print(df[1, ])
  tmdbId title          genres
1      2 Ariel Comedy|Drama|Romance|Crime
                                overview
1 A Finnish man goes to the city to find a job after the mine where he worked is closed and his father commits suicide.
                                poster_url vote_average vote_count popularity runtime release_date
1 https://image.tmdb.org/t/p/w500/ojDg0PGvs6R9xVFodRct2kdI6wC.jpg          7.104          369          2.5706          73 1988-10-21
>
```

Data cleaning movie dataset

After running the extraction script for a period of time, a dataset containing more than 1.3 million movie records was generated. However, during the initial analysis, it was observed that the dataset contained a significant amount of missing data and noise.

In particular, many entries had incomplete information in key fields, and a large number of movies were of very low relevance for recommendation purposes, such as films without a poster image or movies with only one vote or less on IMDb. These records negatively affected the quality and reliability of the dataset.

To address this, a data cleaning process was implemented using a dedicated R script (`datacleaning_movies.R`). This script performed a series of filtering steps:

- Removed movies without a valid poster (`poster_url` missing or empty).
- Filtered out movies with very low engagement, specifically those with 1 vote or less.
- Detected and removed records with missing or empty values across any of the key fields.
- Ensured that only fully complete and consistent records were kept in the final dataset.

After applying these filters, the dataset was significantly reduced but became much more consistent and reliable for recommendation purposes. Finally, the cleaned dataset was exported to a new CSV file with 243426 records for use in the recommendation system.

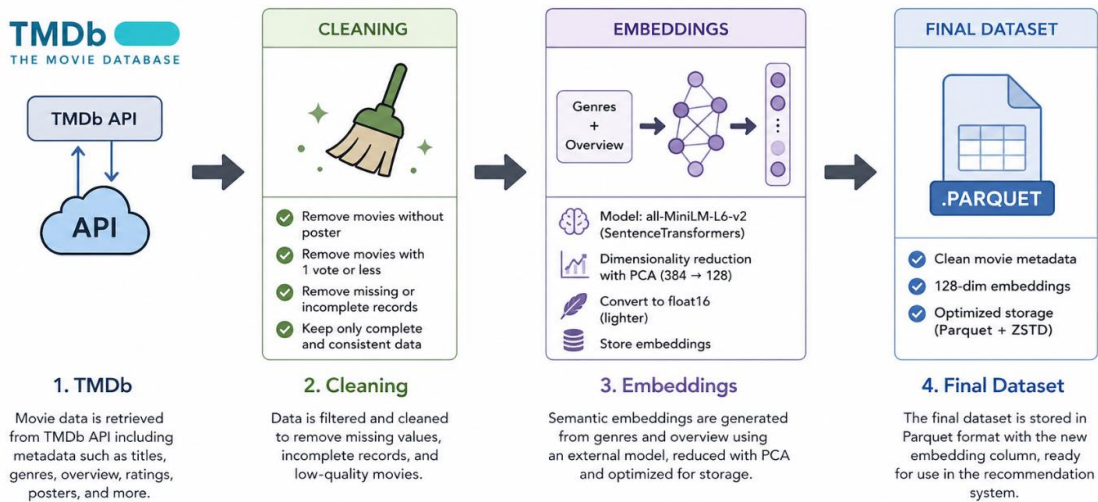
```
Películas sin poster: 0
>
> df <- df %>%
+   filter(!is.na(posters_url) | posters_url == "")
>
> cat("Quedan tras quitar sin poster:", nrow(df), "\n\n")
Quedan tras quitar sin poster: 243428
>
> #registros con 1 voto o menos
> df_1_voto <- df %>%
+   filter(vote_count <= 1 | is.na(vote_count))
>
> cat("Películas con 1 voto o menos:", nrow(df_1_voto), "\n")
Películas con 1 voto o menos: 0
>
> df <- df %>%
+   filter(vote_count > 1)
>
> cat("Quedan tras quitar películas con 1 voto:", nrow(df), "\n\n")
Quedan tras quitar películas con 1 voto: 243428
>
> #campos
> df_campos_faltantes <- df %>%
+   filter(if_any(everything(), ~ is.na(.) | . == ""))
>
> cat("Películas con algún campo faltante:", nrow(df_campos_faltantes), "\n")
Películas con algún campo faltante: 0
>
> df_totalmente_limpio <- df %>%
+   filter(if_all(everything(), ~ !is.na(.) & . != ""))
>
> cat("Quedan tras limpieza completa:", nrow(df_totalmente_limpio), "\n\n")
Quedan tras limpieza completa: 243428
>
> #resumen
> total_final <- nrow(df_totalmente_limpio)
>
> cat("TOTAL FINAL:", total_final, "\n")
TOTAL FINAL: 243428
> cat("TOTAL ELIMINADO:", total_inicial - total_final, "\n\n")
TOTAL ELIMINADO: 0
>
> # guardad csv
> ruta_salida <- "C:/Users/Jordi/Desktop/tmdb_dataset_totalmente_limpio.csv"
>
> write.csv(df_totalmente_limpio, ruta_salida, row.names = FALSE)
>
> cat("Archivo guardado en:", ruta_salida, "\n")
Archivo guardado en: C:/Users/Jordi/Desktop/tmdb_dataset_totalmente_limpio.csv
>
```

Embeddings MovieDataset

To improve the recommendation quality, a new embeddings column was created for the movie dataset. The goal was to capture semantic similarity between films, not only through structured metadata such as genres, but also through the descriptive content of each movie.

For this purpose, the project used an external pretrained model from SentenceTransformers, specifically all-MiniLM-L6-v2. This model was chosen because it provides a good balance between quality, speed, and efficiency, which made it suitable for processing a large dataset of movies. The embeddings were generated from a combined text representation built using the genres and the

overview of each movie. By joining both fields into a single text string, the model was able to learn richer semantic relationships between films.



The embedding generation process was implemented in a dedicated script, which first loaded the cleaned movie dataset, built the combined text for each record, and then generated a vector representation for every movie.

The original embedding size produced by the model was 384 dimensions. However, storing this full representation for more than 240,000 movies would have increased the storage requirements significantly and reduced system efficiency.

To address this, PCA (Principal Component Analysis) was applied to reduce the dimensionality of the embeddings from 384 to 128 dimensions. This step preserved the most relevant semantic information while significantly reducing memory usage and computational cost during similarity calculations.

After dimensionality reduction, the embeddings were converted to float16, further reducing memory consumption while maintaining sufficient precision for similarity-based recommendations.

Finally, the dataset was stored in Parquet format using ZSTD compression, which was selected for its high compression ratio and fast read performance. This decision was important to ensure that the system could load and process embeddings efficiently during recommendation generation.

Nombre	Tamaño	Tipo
tmdb_dataset_full	104.411 KB	Microsoft Excel Comma Separated Values File
tmdb_embeddings.parquet	101.307 KB	Archivo PARQUET

As a result, the final movie dataset included an additional embedding column, which became a core component of the hybrid recommendation system. These embeddings allow the model to recommend movies not only based on genres or popularity, but also on deep semantic similarity between movie descriptions.

Despite adding a high-dimensional vector representation for each movie, the final storage impact was carefully optimized. The original CSV dataset without embeddings had a size of approximately 104 MB, while the final Parquet file containing embeddings reached only 101 MB, even though it included the same number of records and an additional large vector column per entry. This reduction in size was achieved thanks to the combination of PCA dimensionality reduction (384 → 128), float16 compression, and Parquet + ZSTD encoding, which significantly improved storage efficiency while preserving the semantic quality of the embeddings.

User Dataset

For this movie recommendation project, the dataset was generated from scratch using a custom script called **userdatagenerator.py**, instead of relying on publicly available user datasets. This decision was taken because existing datasets were not sufficiently reliable for the specific needs of the recommendation system. Many of them were incomplete, inconsistent, outdated, or designed for different recommendation approaches, which made them unsuitable for building accurate and meaningful user profiles.

Creating the dataset manually allowed full control over its structure and ensured consistency across all records. It also made it possible to define each feature explicitly according to its role in the recommendation logic.

The dataset includes the following columns:

- **user_id**: A unique identifier assigned to each user. This is essential to distinguish users internally within the system and to link their interactions and preferences without ambiguity.
- **username (unique)**: A unique username used for user identification in the application layer. It ensures that each user can be referenced in a human-readable way while preventing duplicates.
- **password**: Stored for authentication purposes. Although not used in the recommendation logic, it is required for user account management and system access control.

- **email (unique):** A unique email address associated with each user. It is used for account validation, communication, and ensuring that each account corresponds to a single real or simulated user identity.
- **gender:** Included as a demographic feature that can help analyze preference patterns and potentially enrich future recommendation strategies, even if it is not a primary driver of the current model.
- **age:** Another demographic attribute used to provide additional context about user behavior and preferences. It can help identify trends across different age groups.
- **favorite_genres:** This is the most important feature by far for the recommendation model. It directly represents user preferences and is the primary input used to generate personalized movie recommendations. The model heavily relies on this field to match users with relevant content.
- **created_at:** Stores the timestamp of when the user was created. This is mainly used for tracking, sorting, and potential temporal analysis of user growth or activity over time.

Additionally, synthetic users were generated using the **userdatagenerator.py** script to simulate realistic behavior patterns and ensure that the recommendation system could be tested under controlled and reproducible conditions.

The dataset has this aspect:

	user_id	username	email	password	gender	age	favorite_genres	created_at
1	1	user1	user1@example.com	pass1	other	65	Music War Animation	2025-11-24 20:42:47
2	2	user2	user2@example.com	pass2	female	31	Crime Romance Action War	2025-08-10 20:42:47
3	3	user3	user3@example.com	pass3	male	49	Crime Documentary Music TV Movie	2025-08-21 20:42:47
4	4	user4	user4@example.com	pass4	male	49	Comedy	2025-05-05 20:42:47
5	5	user5	user5@example.com	pass5	female	59	Adventure Horror	2026-03-07 20:42:47
6	6	user6	user6@example.com	pass6	female	57	Adventure Documentary Animation Fantasy	2025-11-04 20:42:47
7	7	user7	user7@example.com	pass7	female	59	Horror War	2025-11-16 20:42:47
8	8	user8	user8@example.com	pass8	male	64	Thriller Adventure Action Science Fiction	2025-04-17 20:42:47
9	9	user9	user9@example.com	pass9	other	69	Romance	2025-10-20 20:42:47
10	10	user10	user10@example.com	pass10	male	18	Drama Family History	2025-04-06 20:42:47
11	11	Paco63	paco63@gmail.com	123456	Female	31	Drama	2026-04-08 17:36:47
12	12	Paco69	paco69@gmail.com	123456	Hombre	0	Crime Documentary Action Drama Family	2026-04-19 18:22:45
13	13	Paco697	paco697@gmail.com	123456	Hombre	31	Thriller Adventure Action Romance	2026-04-19 18:25:35
14	14	XiaoXiao	xiaoxiao@gmail.com	123456	Female	22	Animation	2026-04-22 17:36:38

User-Movie Interactions Dataset

For this movie recommendation project, a separate dataset was created to store user–movie interactions. This is a critical component of the system, since recording each interaction between users and movies is essential for building accurate user profiles later on. Without this layer of behavioral data, the recommendation engine would lack real evidence of user preferences.

This dataset, like the user dataset, was generated using a custom script called **userinteraction_generator.py**. This approach was chosen to ensure full control over the structure, consistency, and quality of the generated data. Instead of relying on external datasets, which often contain noise, missing values, or inconsistent formats, generating the data programmatically allows the system to precisely match the requirements of the recommendation logic.

The dataset was also intentionally designed as a separate structure from the user dataset. This separation is a better architectural decision because it avoids data redundancy, improves scalability, and clearly distinguishes static user information from dynamic behavioral data. While the user dataset defines who the user is, this interaction dataset captures how the user behaves over time with different movies, which is fundamental for recommendation models.

The dataset includes the following columns:

- **interaction_id**: Unique identifier for each interaction record. It ensures that every user–movie interaction is uniquely traceable and prevents ambiguity when handling or updating specific entries.
- **user_id**: Identifier of the user who performed the interaction. This column is essential to link each rating or action to a specific user and is the key connection point with the user profile dataset.
- **movie_id**: Identifier of the movie involved in the interaction. It allows the system to know exactly which movie was rated or evaluated, and connects the interaction data with the movie catalog.
- **rating**: Numerical score given by the user to a movie. This is one of the most important features in the dataset, as it directly represents user preference and is a primary signal used by the recommendation model.
- **valuation_date**: Timestamp indicating when the interaction occurred. This field enables temporal analysis of user behavior and can help track how preferences evolve over time.

Overall, this dataset plays a fundamental role in the recommendation pipeline, as it provides the behavioral foundation required to infer user preferences and generate personalized movie recommendations.

The dataset has this aspect:

data >  interactions.csv

	interaction_id	user_id	movie_id	rating	valoration_date
1	1	1	967738	3.5	2025-08-31 21:04:07
2	2	1	1222169	5.0	2025-09-19 21:04:07
3	3	1	253872	4.5	2025-07-06 21:04:07
4	4	1	1187028	4.0	2025-12-23 21:04:07
5	5	1	929182	4.5	2025-09-03 21:04:07
6	6	1	955084	4.5	2025-04-09 21:04:07
7	7	1	1168508	5.0	2025-06-21 21:04:07
8	8	1	1496380	4.5	2025-10-25 21:04:07
9	9	1	506530	4.0	2025-09-04 21:04:07
10	10	1	442124	3.0	2026-03-31 21:04:07
11	11	1	1254130	5.0	2025-11-08 21:04:07
12	12	1	1328292	5.0	2025-04-21 21:04:07
13	13	1	696820	2.0	2025-08-16 21:04:07
14	14	1	1174708	0.5	2025-04-16 21:04:07
15	15	1	1553117	3.0	2025-12-18 21:04:07
16	16	2	139608	1.5	2025-09-17 21:04:07
17	17	2	399700	1.5	2025-06-10 21:04:07

Scalability of the Storage System

The storage architecture is designed to be scalable by separating the system into independent datasets that grow at different rates. This avoids bottlenecks and allows each component to scale according to its own data dynamics.

The **interaction dataset** is the most dynamic and will grow continuously as users generate new ratings. The **movie dataset**, obtained through an API-based extraction pipeline, can be expanded incrementally over time. The **user dataset** remains relatively stable, as user profiles do not change as frequently.

This separation ensures that increases in data volume do not affect the overall structure of the system. It also allows efficient processing, easier maintenance, and the ability to extend the system in the future without requiring major architectural changes.

High availability of the storage system

In the current architecture, data persistence is based on local files (Parquet and CSV) and a local cache directory. While this approach is simple and efficient for development and small-scale deployment, it does not provide true high availability in a production sense.

From a realistic perspective, the storage layer represents a single point of failure. All critical components—movie metadata with embeddings, user data, interaction logs, and precomputed caches—are stored on a single filesystem. If the disk becomes unavailable, corrupted, or the host system fails, the recommender service would lose access to essential data and would not be able to operate correctly.

There is also no built-in redundancy or replication mechanism. Data is not duplicated across multiple nodes, nor is there a distributed storage system (such as object storage or a replicated database). This means the system cannot tolerate hardware failure or support seamless failover to another instance.

Additionally, the cache layer is local to the application instance. This prevents horizontal scaling with multiple service replicas, since each instance would maintain its own isolated cache state, leading to inconsistencies and reduced reliability under load balancing scenarios.

In summary, the current storage design provides acceptable reliability for a single-node environment, but it does not meet high availability standards. To achieve true high availability, the system would require:

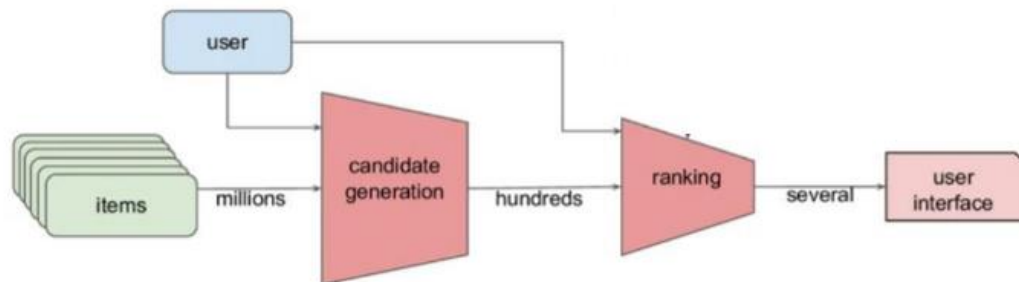
- A distributed or replicated storage solution (e.g., S3-compatible object storage or a distributed database)

- Separation between storage and compute layers
- Shared or distributed caching (e.g., Redis cluster)
- Redundancy and backup strategies with automatic failover capabilities

Without these improvements, the system remains vulnerable to downtime in case of infrastructure failure.

MovieRecommender

The main idea is to filter thousands of movies based on the user's preferences to obtain a smaller set of movies that the user is likely to enjoy. Then, a more precise ranking is applied to those candidate movies in order to generate the best recommendations from that set.



The recommendation engine is implemented through a `MovieRecommender` class that centralizes the full recommendation workflow. Its design follows a clear pipeline: first, the system loads the movie catalogue, user information, interaction history, and precomputed embeddings; then it builds a user profile; afterwards, it generates a candidate pool of movies; and finally, it ranks those candidates according to different recommendation strategies.

The model is not based on a single signal. Instead, it combines several sources of information: explicit user preferences stored in the users dataset, observed behavior from the interactions dataset, and rich movie metadata together with semantic embeddings from the movie dataset. This makes the system hybrid, because it mixes content-based similarity, profile-based filtering, and popularity-driven ranking.

A key part of the design is the use of a cache. The movie metadata is transformed into a precomputed cache file that stores derived features such as normalized popularity, vote confidence, weighted rating, and a final catalogue score. This reduces repeated processing and makes the system faster and more efficient during runtime. The cache is rebuilt only when necessary, which helps keep the application responsive.

The recommender also uses an `EmbeddingIndex`, which stores movie embeddings in a matrix form together with an ID-to-index mapping. This allows the system to compare movies semantically using cosine similarity, which is especially useful for personalized recommendations and “similar movie” retrieval.

Candidate generation

Candidate generation is the first filtering stage of the recommendation process. Its purpose is to reduce the full movie catalogue into a smaller and more relevant set of possible recommendations before ranking is applied.

To build this candidate pool, the system first creates a user profile. This profile includes watched movies, positive and negative feedback, favorite genres, genre preferences learned from interaction history, and a user embedding vector built from previously liked and disliked films.

Once the profile is available, the system removes movies already watched by the user. After that, it assigns an initial score to each remaining movie using three main components: catalogue quality, genre affinity, and embedding similarity. The catalogue score comes from precomputed movie quality and popularity signals. Genre affinity measures how well the movie genres match the user's preferences. Embedding similarity measures semantic closeness between the user profile vector and the movie embedding.

This step is important because it limits the search space to the most promising titles. Instead of ranking the entire dataset, the model works only with a smaller candidate set, which improves both performance and recommendation relevance.

Ranking

After candidate generation, the system applies a ranking stage. This is where the final order of movies is determined. The recommender supports several ranking modes, each designed for a different use case.

The **balanced** mode is the most personalized one. It combines genre affinity, quality score, popularity, release year, and embedding similarity. It is designed to produce recommendations that are both relevant to the user and reasonably strong in overall quality.

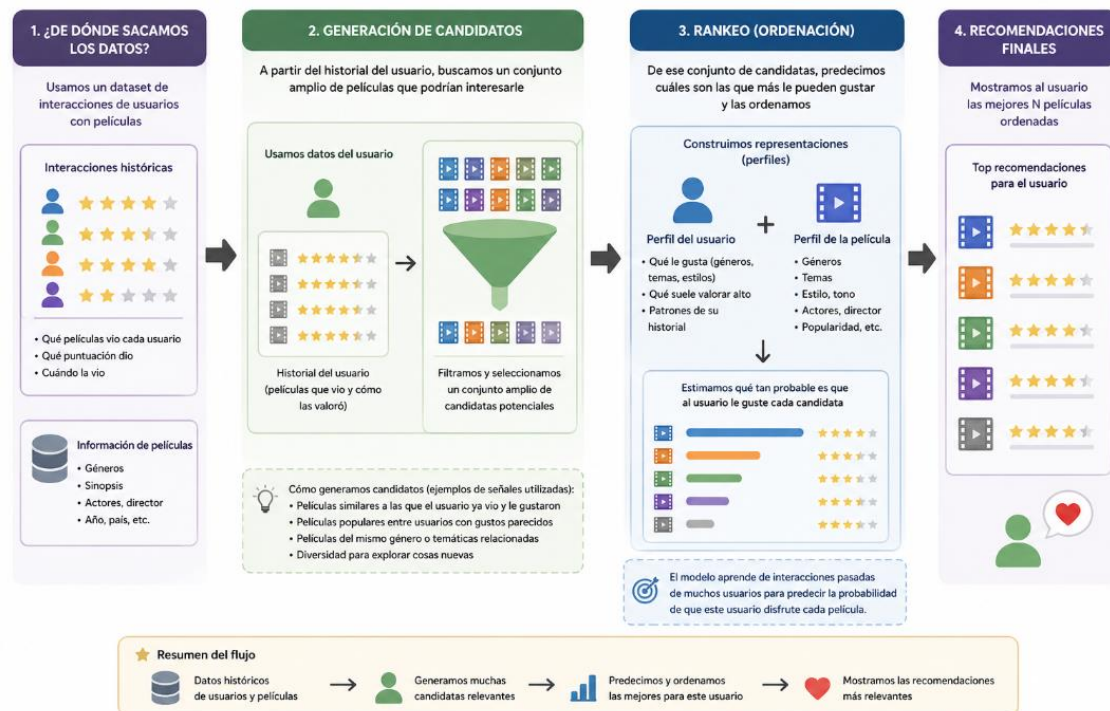
The **popular** mode gives more weight to global audience signals such as vote count, popularity, and weighted rating. This mode is useful when the goal is to surface well-known or trending titles while still keeping some personalization through genre and embedding signals.

The **random** mode is used to introduce variety. It selects movies from the available pool without applying a detailed ranking formula, while still avoiding titles the user has already seen. This is useful for exploration and for preventing the interface from becoming too repetitive.

The **similar** mode is item-to-item rather than user-to-item. It compares the reference movie with the rest of the catalogue using genre overlap, shared genres,

title similarity, year proximity, runtime similarity, and embedding similarity. This mode is especially useful on the movie detail screen, where the goal is to recommend films related to the one the user is currently viewing.

In all ranking modes, already watched movies are penalized or removed so that the system does not recommend content the user has already consumed. This ensures that the output remains useful and personalized.



Informes de monitorización y análisis de rendimiento del sistema

Monitoring Reports and System Performance Analysis

Evaluating the quality of a recommender system is not straightforward. In this project, we do not rely solely on classical offline metrics, because those metrics do not fully capture whether the recommendations are actually coherent, explainable, and aligned with the user profile. For that reason, we created an audit page that acts as a monitoring and inspection layer for the whole recommendation pipeline.

This page allows us to inspect three key dimensions of the system: the user profile, the main recommendation list, and the popularity-based baseline. By exposing the internal score breakdown, the page makes it possible to verify how each recommendation is produced and to identify whether the ranking logic behaves as expected. In practice, this has been extremely useful for comparing different weight configurations and for tuning the model iteratively.

Because the system does not yet have a large-scale ground-truth evaluation framework with explicit user feedback, the adjustment of the scoring weights has been done through empirical calibration. That is, we have been refining the contribution of each signal by observing the audit metrics and checking how they affect the final ordering of the recommendations. This manual tuning approach is not arbitrary: it is a controlled, evidence-driven process based on the behavior of the system in the audit interface. The goal is to balance relevance, diversity, popularity, and semantic similarity without overfitting to a single signal.

The audit dashboard also helps us understand the impact of embeddings on the final score. By comparing rankings with and without embedding contributions, we can measure how much semantic similarity changes the recommendation order. This comparison is especially valuable because embeddings often introduce meaningful improvements that are not visible through simple popularity-based metrics alone.

System Performance and Caching Strategy

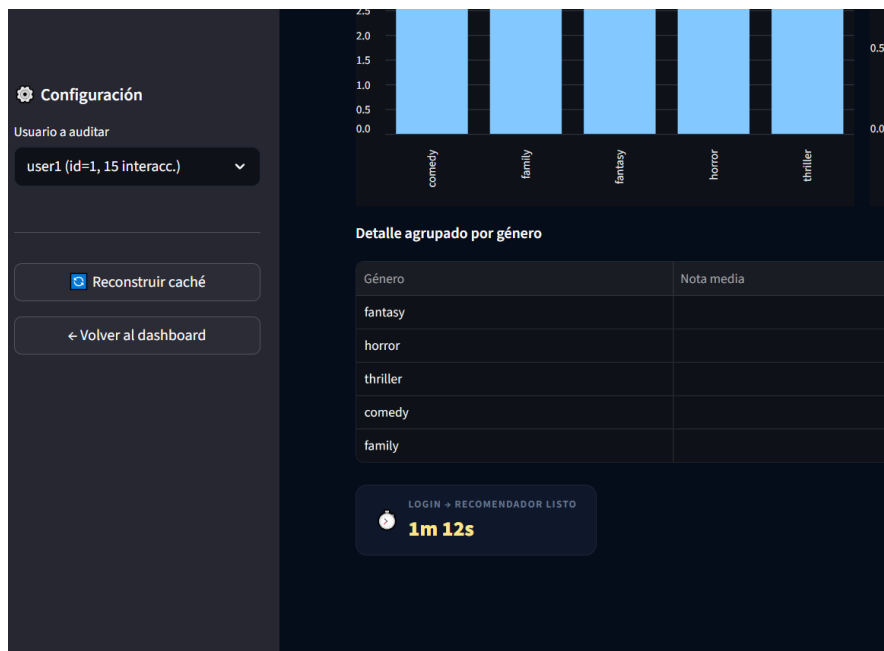
To ensure the system remains responsive during recommendation generation and auditing, a caching layer has been implemented for dataset preprocessing. This is necessary because the pipeline involves expensive operations such as feature engineering, embedding indexing, and aggregation of user interaction data.

Instead of recomputing all derived features on each run, the system stores a precomputed version of the enriched movie dataset in a persistent Parquet-based cache. The recommender then loads this cached representation directly, significantly reducing initialization time and improving overall throughput. Embeddings are also indexed once and reused across requests, avoiding repeated similarity computations.

Additionally, a performance monitoring field has been added to measure the time required to generate recommendations end-to-end. This makes it possible to directly observe the impact of caching: without it, recommendation generation is noticeably slower due to full recomputation of features; with caching enabled, latency is significantly reduced, enabling interactive use of the audit system.

Overall, this approach reduces computational overhead, improves scalability, and provides a clear and measurable performance benefit while keeping the pipeline consistent and reproducible.

Before the cache system implemented:



After:

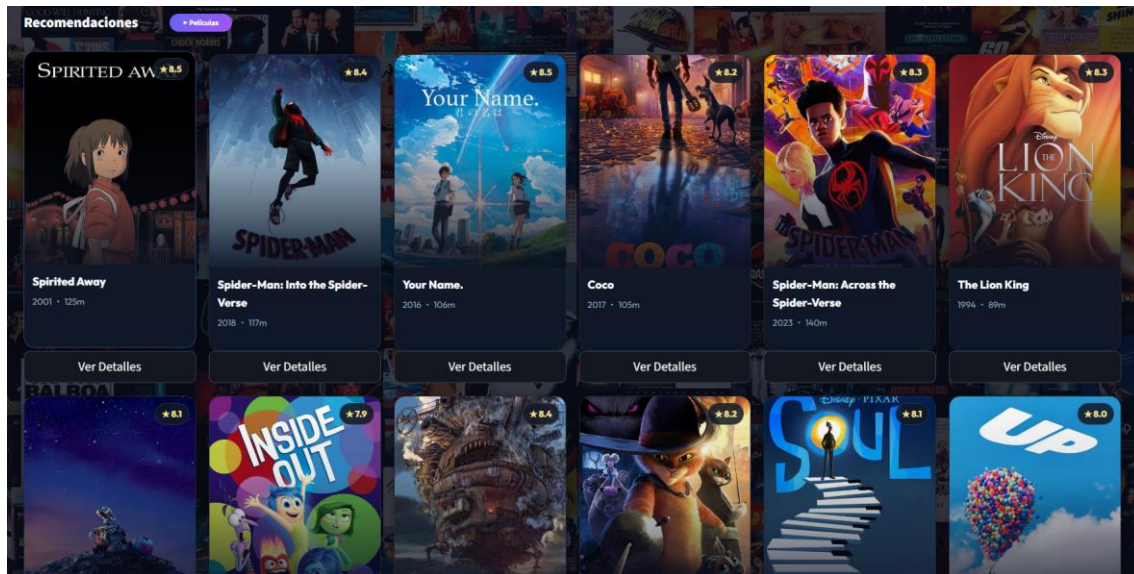


Example Of Usage

To demonstrate how the recommendation system adapts dynamically to user behavior, a simple experimental scenario was created.

First, a new user profile was generated with Comedy as the only favorite genre in the user dataset. At this stage, the user had no interaction history, meaning the recommendation system could only rely on the static preference defined in the profile.

When entering the application for the first time, the recommendations shown on the main page were mainly popular animation and comedy-related movies. This occurs because the recommendation engine initially prioritizes the user's favorite genres combined with the catalogue's popularity and quality scores. Since animation films often overlap with comedy in genre classification and tend to have high popularity scores, they appeared frequently among the first recommendations.



In the next step, a behavioral change was simulated to evaluate how the system reacts to new user interactions. Using R, the dataset was analyzed to extract the 50 horror movies with the highest number of votes. These movies were selected because they represent well-known and widely rated films within the horror genre.

```

Top 50 Horror tmdbId:
> print(top_horror_ids)
[1] 346364 694 419430 381288 72190 348 44214 447332 19908 138843 578 539 405774 176 474350 126889 300668 747 259693
[20] 158015 333371 9552 22970 493922 170 396535 345940 458723 530385 300669 561 395992 1091 4232 282035 310131 2668 520763
[39] 49018 439079 1933 60304 381283 927 270303 1576 238636 1813 36647 565

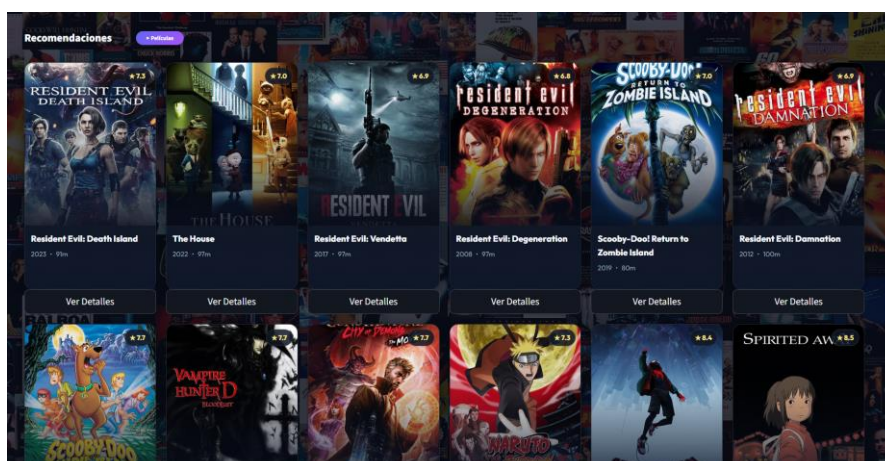
```

After obtaining these movie IDs, a Python script was used to generate 50 new interaction records for the previously created user. Each of these interactions corresponded to one of the selected horror movies and was assigned the maximum rating of 5 stars, indicating a very strong positive preference. As a result, the user's entire interaction history consisted of highly positive ratings for horror movies.

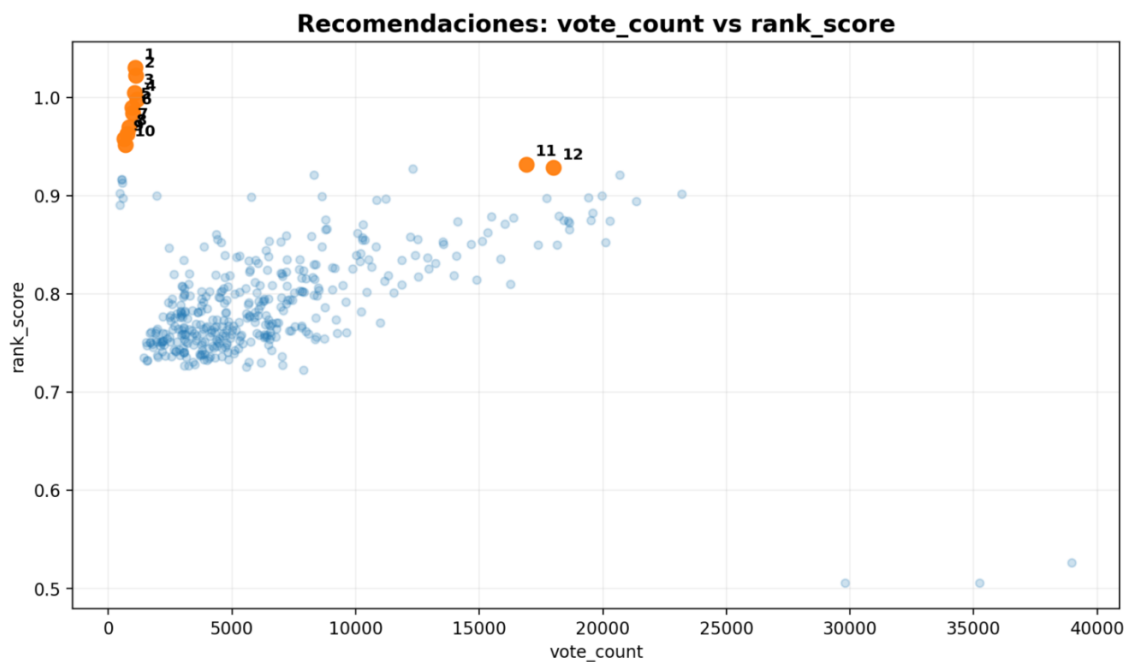
	interaction_id	user_id	movie_id	rating	valoracion_date
243	243	23	345940	5.0	2025-10-20 23:35:09
244	244	23	458723	5.0	2026-02-06 23:35:09
245	245	23	530385	5.0	2025-11-22 23:35:09
246	246	23	300669	5.0	2026-02-10 23:35:09
247	247	23	561	5.0	2026-04-26 23:35:09
248	248	23	395992	5.0	2025-10-24 23:35:09
249	249	23	1091	5.0	2026-01-24 23:35:09
250	250	23	4232	5.0	2025-09-06 23:35:09
251	251	23	282035	5.0	2025-11-07 23:35:09
252	252	23	310131	5.0	2025-07-16 23:35:09
253	253	23	2668	5.0	2025-09-24 23:35:09
254	254	23	520763	5.0	2026-04-07 23:35:09
255	255	23	49018	5.0	2025-09-09 23:35:09
256	256	23	439079	5.0	2025-11-29 23:35:09
257	257	23	1933	5.0	2025-12-18 23:35:09
258	258	23	60304	5.0	2025-10-27 23:35:09
259	259	23	381283	5.0	2025-12-16 23:35:09
260	260	23	927	5.0	2025-05-06 23:35:09
261	261	23	270303	5.0	2025-08-31 23:35:09
262	262	23	1576	5.0	2026-01-16 23:35:09
263	263	23	238636	5.0	2026-03-16 23:35:09
264	264	23	1813	5.0	2025-05-17 23:35:09
265	265	23	36647	5.0	2025-05-21 23:35:09

Once these interactions were inserted into the interactions dataset, the application was launched again and the recommendation page was revisited.

At this point, the system produced noticeably different recommendations. Although animation and comedy movies still appeared, the system started to recommend a significant number of horror movies as well. This behavior occurs because the recommendation model builds a dynamic user profile that incorporates both the initial favorite genres and the observed interaction history. Since 100% of the user's ratings were highly positive for horror movies, the system interprets this as a strong signal that the user enjoys this genre.



Additionally, it is important to note a limitation introduced by the way the experiment was simulated. Since the user interactions were generated using the 50 most popular horror movies, those films were already marked as watched/rated by the user. The recommendation system penalizes or excludes previously interacted movies when generating new recommendations. As a consequence, many of the most popular horror movies cannot appear in the recommendation list, even though the user clearly showed a strong preference for that genre. Instead, the system tends to recommend other horror movies that the user has not rated yet, or popular animation movies that still achieve a sufficiently high score due to their strong popularity metrics and the genre match with the user's initial profile. This explains why some highly popular animation titles may still appear among the recommendations despite the user's strong positive interactions with horror films.



Conclusions y future Improvements

Conclusions

This project successfully achieved its main goal: to design and develop a movie recommendation system capable of offering personalized suggestions based on user preferences, ratings, and interactions. The final solution combines different sources of information, such as movie metadata, user favorite genres, and interaction history, to generate recommendations that are more relevant and dynamic than a traditional generic approach.

One of the most important results of the project is the hybrid nature of the recommender. Instead of relying on a single criterion, the system uses several signals to improve recommendation quality. It takes into account the user's preferences, the movies they have already watched, the similarities between films, and global popularity indicators. This makes the system more flexible and better adapted to different recommendation scenarios.

Another relevant conclusion is that the system is functional and scalable at a project level, but it still has room for improvement in terms of architecture, performance, and user experience. These limitations open the door to several future enhancements.

Future Improvements

Although the current version of the system is fully usable, several improvements could be implemented in future versions to make it more robust, scalable, and professional.

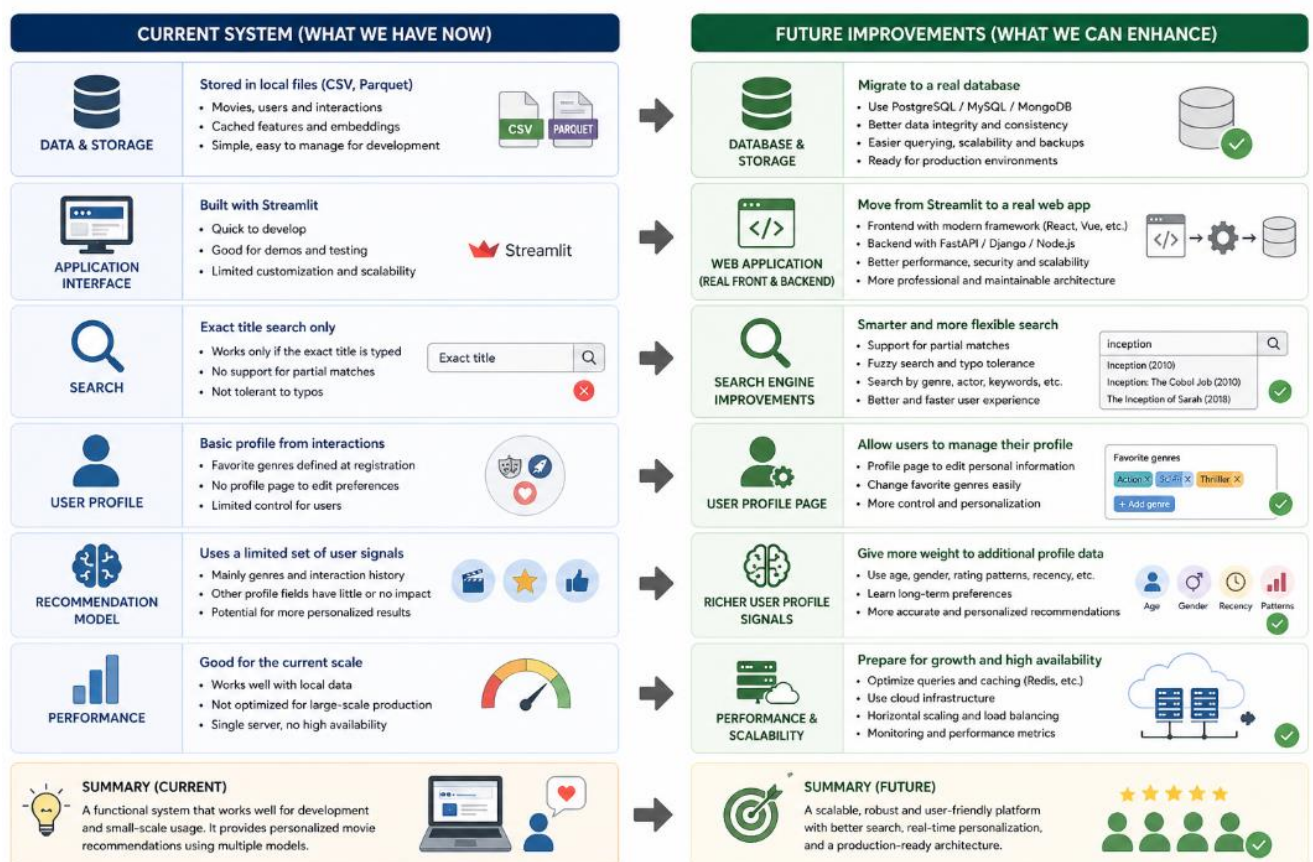
First, the storage layer could be migrated from local datasets and files to a real database management system. This would improve data consistency, querying capabilities, and scalability, especially if the platform grows in number of users, movies, and interactions. A database-based architecture would also make the system more suitable for production environments.

Second, the current Streamlit-based interface could be replaced by a real front-end and back-end architecture. A dedicated web application would allow better separation of concerns, greater flexibility in the design of the user interface, and easier integration with authentication, user profiles, and recommendation services.

Another important improvement would be to enhance the search system. At the moment, the search functionality is too strict, since it only works correctly when the exact movie title is entered. In future versions, the search engine should support partial matches, spelling variations, and more intelligent text matching so that users can find movies more easily.

It would also be useful to add a user profile page where users can manage their information and modify their favorite genres directly from the interface. This would improve personalization and give users more control over the recommendation process.

Finally, the recommendation model could be improved by giving more weight to additional user profile fields. At present, the system mainly focuses on favorite genres and interaction history, but other information such as age, gender, and long-term rating patterns could also be used to refine the recommendations further. This would allow the system to build richer user profiles and generate even more accurate suggestions.



Demonstration videos

Functional video

<https://www.youtube.com/watch?v=Nn9h0Bcgy1s>

Performance analysis video

<https://www.youtube.com/watch?v=mhadwNg9-I0>